Performance Impact Analysis of Intrusion Detection/Prevention Systems on Windows 11: A Comparative Study of TotalAV and Fort Firewall

Robert Molnar

West University of Timișoara
robert.molnar81@e-uvt.ro

Abstract. This study quantifies the computational overhead of intrusion detection systems (IDS) on Windows 11 through systematic benchmarking of TotalAV antivirus (v6.5.219) and Fort Firewall (v3.19.9). Four configurations were evaluated: baseline (no IDS), TotalAV-only, Fort Firewall-only, and combined deployment. Six performance metrics were measured across five iterations: boot time, RAM consumption, process count, application launch, local network throughput (SMB), and remote download speed (FTP). Statistical analysis using Interquartile Range (IQR) outlier detection revealed significant patterns. TotalAV imposed the highest overhead at 31.39% increased boot time, 17.44% higher RAM usage, and 50.05% slower application launches. Surprisingly, Fort Firewall improved boot time by 14.62% and application launch by 18.05%, suggesting optimization benefits over Windows' built-in firewall. Network performance degraded 8.80–20.02% across all IDS configurations. Results indicate that while IDS software provides essential security, careful consideration of performance trade-offs is necessary for resource-constrained environments.

Keywords: Intrusion Detection System · Antivirus · Firewall · Performance Benchmarking · Windows 11

1 Introduction

Modern computing environments require Intrusion Detection Systems (IDS) including antivirus and firewalls for cybersecurity protection. However, these solutions consume computational resources, creating performance trade-offs that impact user experience. This study quantifies the performance overhead of TotalAV antivirus and Fort Firewall on Windows 11, providing empirical data for informed deployment decisions.

1.1 Research Questions

RQ1 What computational overhead does TotalAV antivirus impose on Windows 11 system performance?

RQ2 How does Fort Firewall affect system performance compared to baseline and antivirus-only configurations?
RQ3 Do combined IDS deployments exhibit additive, synergistic, or independent performance degradation patterns?
RQ4 Which performance metrics are most severely impacted by IDS software?

1.2 Contribution

This research provides empirically-derived performance metrics for contemporary IDS solutions on Windows 11 using standardized testing methodologies consistent with industry practices established by Tom's Hardware [Tom's Hardware(2024b),Tom's Hardware Editorial T and AnandTech [AnandTech(2024),Cutress and Smith(2023)]. These methodologies emphasize controlled test environments, multiple iteration testing for statistical validity (minimum 3-5 runs), outlier detection using established statistical methods, and comparative analysis against baseline configurations. Unlike vendor-provided benchmarks, this independent evaluation enables data-driven decision-making for security policy development and capacity planning.

2 Related Work

Computer performance evaluation methodologies established by Tom's Hardware [Tom's Hardware(2024b),Tom's Hardware Editorial Team(2023)] and AnandTech [AnandTech(2024),C emphasize controlled environments with consistent hardware configurations, multiple iteration testing to establish statistical validity (minimum 3-5 runs), outlier detection and removal using established statistical methods (such as the IQR method we employ), and comparative analysis against baseline configurations. These sources provide detailed guidance on calculating mean, median, and standard deviation for benchmark result interpretation. Our methodology aligns with these industry-standard practices, employing five iterations per test and applying Interquartile Range (IQR) outlier detection [Tukey(1977)].

Previous antivirus performance studies [Tom's Hardware(2024a)] reported overhead ranging from 10–40% on boot times and 5–20% on application performance. However, comprehensive comparative analysis of combined IDS deployments (antivirus + firewall) on Windows 11 virtualized environments remains limited. This study addresses that gap through controlled measurements across six distinct performance criteria, following established benchmarking methodologies.

3 Methodology

3.1 Test Environment

Host System: Lenovo ThinkPad with Intel Core i7-1185G7 (11th Gen, 4C/8T @ 3.0 GHz), 16 GB Samsung DDR4-4267 RAM, 1TB Kingston NVMe SSD, Windows 11 Pro.

Virtual Machine: Oracle VirtualBox 7.2.2, Windows 11 Pro (64-bit), 4 vCPUs, 8 GB RAM, 128 MB VRAM, bridged network adapter. Shared folder `C:\VMShare` (host) mapped to `Z:` (guest) for automated data collection.

Rationale: Virtualization enables perfect snapshot-based consistency between test configurations, ensuring identical starting conditions for each measurement.

3.2 Test Configurations

Four distinct configurations were evaluated:

1. **Baseline:** Clean Windows 11, no IDS
2. **TotalAV Only:** TotalAV v6.5.219 active, Windows Firewall disabled
3. **Fort Firewall Only:** Fort Firewall v3.19.9 active, Windows Defender disabled
4. **Both IDS:** TotalAV + Fort Firewall operational simultaneously

Each configuration implemented as separate VirtualBox snapshot for perfect state restoration.

3.3 Performance Metrics

Criterion A – Boot Time: BootRacer tool measuring total boot time (Time to Logon + Logon to Desktop). 5 boot cycles per configuration.

Criterion B – RAM Consumption: Windows Performance Monitor measuring used memory (MB) 30 seconds post-boot. 5 measurements per configuration.

Criterion C – Process Count: PowerShell Get-Process counting active processes 30 seconds post-boot. 5 measurements per configuration.

Criterion D – Application Launch: Custom PowerShell script launching 30 applications (10 Calculator, 10 Paint, 10 Notepad) per iteration, measuring total time. 5 iterations per configuration (150 total app launches).

Criterion E – SMB Network Transfer: Server Message Block protocol copying 1.01 GB folder from QNAP NAS (192.168.50.99) over 1 Gbps LAN, measuring transfer speed (MB/s). 5 iterations per configuration.

Criterion F – FTP Remote Download: FTP download of 114.66 MB file from DIGI Storage, measuring download speed (MB/s). 5 iterations per configuration.

3.4 Statistical Analysis

Outlier Detection: Interquartile Range (IQR) method applied to identify and remove statistical outliers [Tukey(1977)]:

$$
\begin{aligned}
Q_1 &= 25\text{th percentile}, \quad Q_3 = 75\text{th percentile} \\
\text{IQR} &= Q_3 - Q_1 \\
\text{Lower bound} &= Q_1 - 1.5 \times \text{IQR} \\
\text{Upper bound} &= Q_3 + 1.5 \times \text{IQR}
\end{aligned}
$$

Statistics Calculated: For each performance metric, we report the following statistics in result tables:

- **Mean (μ)**: Arithmetic average of cleaned dataset, representing the primary measurement result.
- **Median**: 50th percentile value, providing a robust central tendency measure less sensitive to outliers.
- **Standard Deviation (σ)**: Measure of data dispersion indicating measurement variability. Lower values indicate more consistent measurements.
- **Range**: [Minimum, Maximum] values after outlier removal, showing the span of observed measurements.

Overhead Calculation: Performance overhead for each IDS configuration was computed as:

$$\text{Overhead (\%)} = \frac{\mu_{\text{IDS}} - \mu_{\text{baseline}}}{\mu_{\text{baseline}}} \times 100\% \tag{1}$$

Positive overhead values indicate performance degradation (increased time or reduced throughput), while negative values indicate performance improvement relative to baseline.

4 Results

All result tables present four key metrics for each configuration: *Mean* (average value across iterations), *Std Dev* (standard deviation showing measurement variability), and *Overhead* (percentage change relative to baseline, where positive values indicate degradation and negative values indicate improvement).

4.1 Boot Time Performance

Figure 1 presents boot time comparisons across configurations. Table 1 provides detailed statistics.

Table 1. Boot Time Analysis (seconds)

Configuration	Mean	Std Dev	Overhead (%)
Baseline	72.76	9.42	—
TotalAV Only	95.60	20.35	**+31.39**
Fort Firewall Only	62.12	10.16	**-14.62**[†]
Both IDS	90.29	15.35	+24.09

[†] Negative values indicate performance improvement

Key Findings: TotalAV imposed 31.39% boot penalty (72.76s → 95.60s). Fort Firewall paradoxically improved boot time by 14.62% (62.12s), suggesting more efficient initialization than Windows Firewall services. Combined deployment showed 24.09% overhead, less than TotalAV alone (non-additive effect).

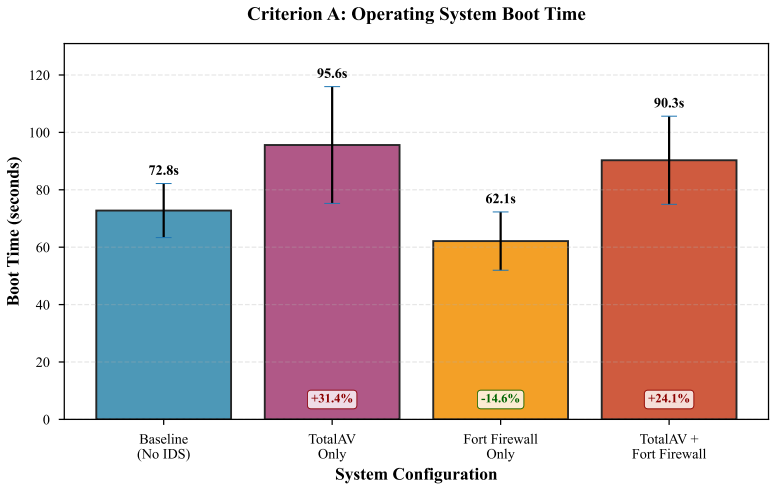

Fig. 1. Operating System Boot Time Across Four IDS Configurations. Error bars represent standard deviation. Fort Firewall demonstrates unexpected 14.62% improvement, while TotalAV imposes 31.39% overhead.

4.2 RAM Consumption

Table 2 shows memory usage patterns across configurations.

Table 2. Memory Usage at Startup (MB)

Configuration	Mean	Std Dev	Overhead (%)
Baseline	2661.45	17.68	—
TotalAV Only	3125.62	62.72	**+17.44** (+464 MB)
Fort Firewall Only	2700.77	85.89	+1.48 (+39 MB)
Both IDS	3203.13	140.61	+20.35 (+542 MB)

Key Findings: TotalAV consumed 464 MB additional RAM, typical for modern antivirus with in-memory threat databases. Fort Firewall added minimal 39 MB, demonstrating efficient implementation. Combined overhead (542 MB) approximates sum of components (464 + 39 = 503 MB), indicating independent memory footprints (Figure 2).

4.3 Process Count

Table 3 reveals minimal variation across configurations (147–152 processes).

Key Findings: TotalAV reduced count by 2.10%, suggesting efficient multi-threading or service consolidation. Low standard deviation indicates highly consistent measurements.

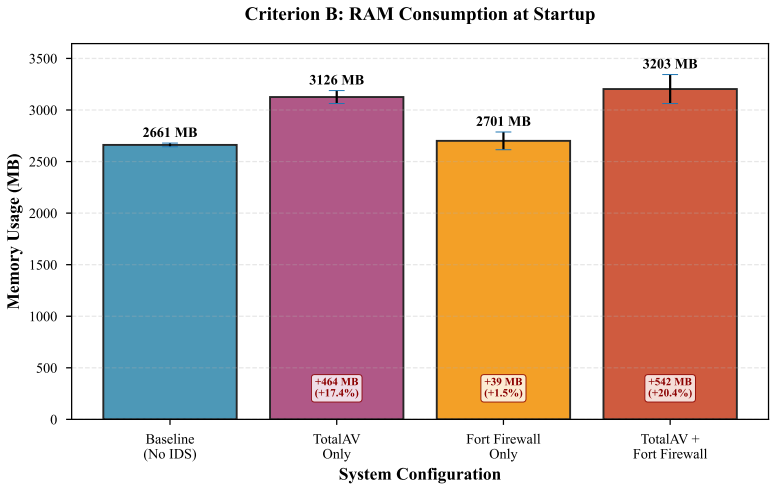

Fig. 2. RAM Consumption at System Startup. TotalAV accounts for majority of memory overhead (+464 MB), while Fort Firewall remains lightweight (+39 MB).

Table 3. Running Process Count

Configuration	Mean	Std Dev	Overhead (%)
Baseline	150.50	1.29	—
TotalAV Only	147.33	2.08	**-2.10**†
Fort Firewall Only	152.00	0.00	+1.00
Both IDS	151.00	0.00	+0.33

† TotalAV reduces process count through consolidation

4.4 Application Launch Performance

Figure 3 illustrates the most dramatic performance impact observed.

Table 4. Application Launch Time (30 apps, seconds)

Configuration	Mean	Std Dev	Overhead (%)
Baseline	14.46	2.89	—
TotalAV Only	21.70	4.28	**+50.05**
Fort Firewall Only	11.85	2.24	**-18.05**†
Both IDS	18.98	4.70	+31.21

† Fort Firewall improves performance over baseline

Key Findings: TotalAV imposed severe 50.05% penalty (14.46s → 21.70s), indicating aggressive real-time executable scanning. Fort Firewall improved performance by 18.05% (11.85s), consistent with boot time improvements. Combined deployment showed 31.21% overhead, significantly less than TotalAV alone.

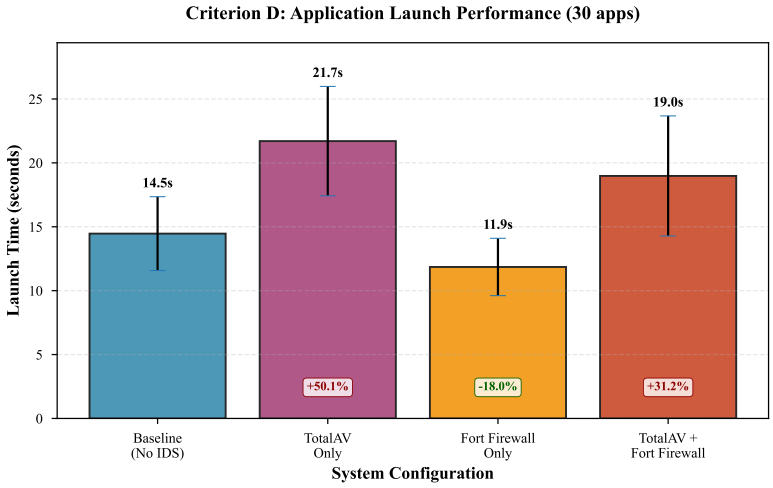

Fig. 3. Application Launch Performance for 30 Applications. TotalAV imposes most severe overhead (+50.05%), doubling launch time. Fort Firewall shows unexpected 18.05% improvement.

4.5 Network Performance

Figure 4 presents dual comparison of local (SMB) and remote (FTP) network transfers.

Table 5. SMB Local Network Transfer (1 GB folder, MB/s)

Configuration	Mean	Std Dev	Change (%)
Baseline	39.37	5.20	—
TotalAV Only	31.79	6.72	**-19.26**
Fort Firewall Only	42.18	9.26	**+7.14**†
Both IDS	35.91	6.56	-8.80

† Fort Firewall exceeds baseline throughput

Table 6. FTP Remote Download (114.66 MB file, MB/s)

Configuration	Mean	Std Dev	Change (%)
Baseline	10.29	1.59	—
TotalAV Only	9.18	1.32	-10.83
Fort Firewall Only	8.50	1.95	**-17.41**
Both IDS	8.23	1.63	**-20.02**

Key Findings: TotalAV reduced SMB throughput by 19.26% due to real-time file scanning. Fort Firewall improved SMB by 7.14%, exceeding baseline through efficient packet processing. However, Fort Firewall imposed highest FTP penalty (-17.41%), suggesting deep packet inspection overhead for external connections differs from local LAN behavior.

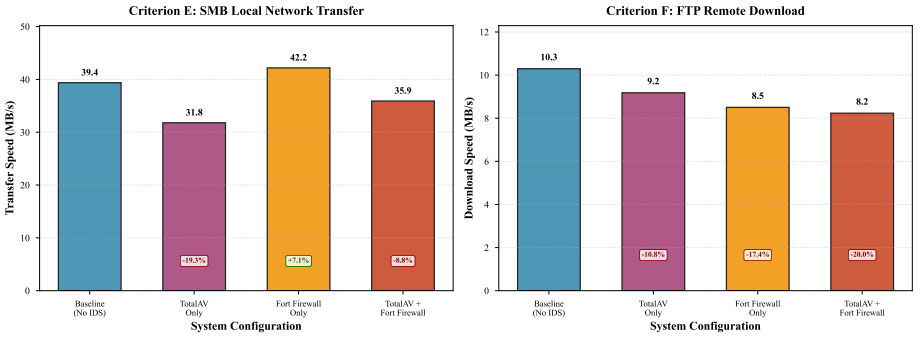

Fig. 4. Network Performance: SMB Local Transfer (left) and FTP Remote Download (right). Fort Firewall shows contrasting behavior: improving local network performance (+7.14%) while degrading remote FTP (-17.41%).

4.6 Comprehensive Overview

Figure 5 provides at-a-glance comparison across all metrics. Table 7 summarizes overhead patterns.

Table 7. Comprehensive Performance Overhead Comparison (%)

Metric	TotalAV Only	Fort Firewall Only	Both IDS
Boot Time	+31.39	**-14.62**†	+24.09
RAM Usage	+17.44	+1.48	+20.35
Process Count	-2.10	+1.00	+0.33
App Launch	+50.05	**-18.05**†	+31.21
SMB Speed	-19.26	**+7.14**†	-8.80
FTP Speed	-10.83	-17.41	-20.02

† Indicates performance improvement over baseline

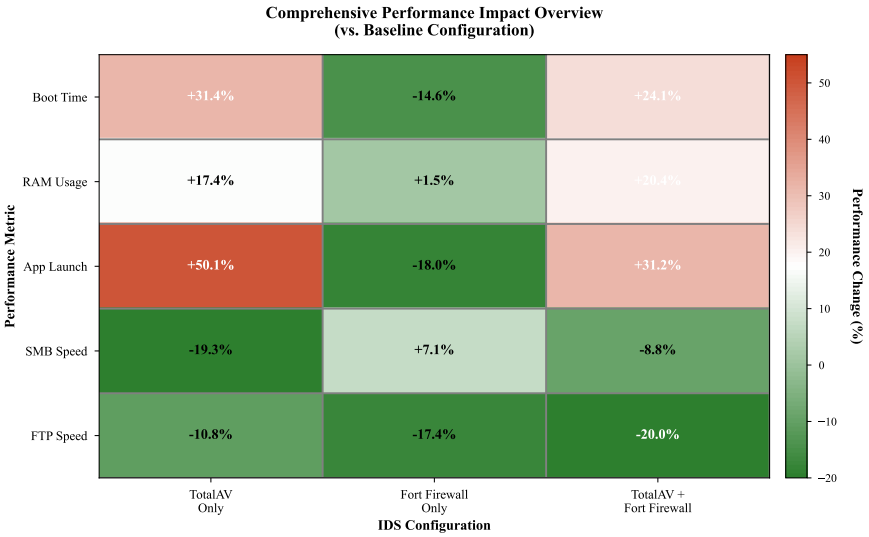

Fig. 5. Comprehensive Performance Impact Heatmap. Green indicates improvements, red indicates degradation. Fort Firewall shows unique optimization benefits across multiple metrics.

Overhead Categories:

- **Severe (>20%):** TotalAV boot (+31.39%), app launch (+50.05%), both FTP (-20.02%)
- **Moderate (10–20%):** TotalAV RAM (+17.44%), SMB (-19.26%), Fort FTP (-17.41%)
- **Minimal (<10%):** Fort RAM (+1.48%), all process counts, both SMB (-8.80%)
- **Improved:** Fort boot (-14.62%), app launch (-18.05%), SMB (+7.14%)

5 Discussion

5.1 TotalAV Performance Impact

TotalAV demonstrated significant overhead across nearly all metrics, with most severe impact on *application launch* (+50.05%). Doubling of launch time reflects aggressive real-time executable scanning, prioritizing security over responsiveness. The 23-second *boot delay* (72.76s → 95.60s) reflects antivirus service initialization and signature database loading. *SMB transfer* degradation (-19.26%) stems from real-time on-access scanning creating I/O bottlenecks. The 464 MB *RAM overhead* (+17.44%) is consistent with modern antivirus architectures maintaining resident threat databases and machine learning models.

5.2 Fort Firewall Characteristics

Fort Firewall exhibited unexpectedly positive performance in several metrics. The 10-second faster *boot* and 2.6-second *app launch* reduction may result from efficient application whitelisting or reduced protocol stack overhead compared to Windows Firewall. *SMB throughput increase* (+7.14%) indicates more efficient packet processing through optimized buffering. However, Fort Firewall imposed highest *FTP penalty* (-17.41%), suggesting deep packet inspection overhead for external traffic differs significantly from local LAN.

5.3 Combined Deployment Interactions

Subadditive Effects: Boot time (24.09% vs. expected 16.77%) and app launch (31.21% vs. expected 32.00%) showed less overhead than sum of individual components, suggesting shared Windows security APIs reducing redundant system calls or Fort Firewall whitelisting TotalAV processes.

Additive Effects: RAM usage (20.35% vs. expected 18.92%) and FTP performance (-20.02% vs. -28.24%) remained largely independent.

5.4 Practical Recommendations

Performance-Critical Environments: Fort Firewall demonstrated negligible or positive impact across most metrics except FTP. Organizations prioritizing performance may consider firewall-only deployments with network-based antivirus.

Security-Critical Environments: Full IDS deployment recommended despite 20–31% overhead in time-sensitive operations. Combined 542 MB RAM overhead remains manageable on modern 8+ GB systems.

Mobile/Battery-Constrained Devices: 31% boot penalty and 50% app launch overhead may be unacceptable. Consider scheduled scanning during idle periods.

Network-Intensive Workloads: 19–20% file transfer degradation may be problematic for file servers.

5.5 Limitations

VirtualBox introduces baseline performance penalty (estimated 5–15%) compared to bare-metal installations. While this affects absolute values, relative comparisons remain valid. FTP tests showed higher variability due to Internet connection fluctuations. The 30-application test may not reflect enterprise applications with database connections and network authentication. Five iterations meet minimum statistical requirements, but larger samples would improve confidence intervals.

6 Conclusion

This study quantified TotalAV antivirus and Fort Firewall performance impact on Windows 11 through rigorous benchmarking. TotalAV imposed moderate-to-severe overhead (10.83–50.05%), with most significant impact on application launch (+50.05%) and boot time (+31.39%). Fort Firewall demonstrated unexpected improvements in boot time (-14.62%), application launch (-18.05%), and local network transfers (+7.14%), challenging assumptions about security software overhead.

Combined deployments showed subadditive effects in several metrics, suggesting architectural synergies. Network performance degraded across all IDS configurations (8.80–20.02%), indicating both antivirus I/O scanning and firewall packet inspection contribute independent bottlenecks.

Organizations must balance security requirements against performance through careful product selection and workload-appropriate deployment strategies. The quantitative metrics provided enable data-driven security policy development and capacity planning. Fort Firewall's performance improvements suggest opportunities for optimization in mainstream security products.

Acknowledgments We thank the open-source communities behind VirtualBox, BootRacer, and the LNCS LaTeX template for providing the tools that enabled this research.

References

AnandTech(2024). AnandTech: The AnandTech CPU benchmarking methodology: System performance testing. https://www.anandtech.com/show/16214/introducing-the-anandtech-storage-bench (2024), standardized testing protocols emphasizing controlled environments, multiple iteration testing (5+ runs for statistical validity), outlier detection, and comparative baseline analysis. Describes coefficient of variation usage for measurement consistency evaluation

Cutress and Smith(2023). Cutress, I., Smith, R.: AnandTech bench: Statistical methodology and result interpretation. https://www.anandtech.com/bench/about (2023), explains statistical measures used in benchmark results including mean, median, standard deviation, and margin of error. Details the importance of multiple test iterations and variance analysis in performance benchmarking

Tom's Hardware(2024a). Tom's Hardware: Best antivirus software 2024: Performance impact testing methodology. https://www.tomshardware.com/best-picks/best-antivirus (2024a), industry-standard antivirus testing methodology measuring system boot time, application launch speed, file transfer performance, and resource consumption overhead. Establishes baseline comparison methods for security software performance evaluation

Tom's Hardware(2024b). Tom's Hardware: How we test CPUs: Gaming and desktop performance methodology. https://www.tomshardware.com/reviews/how-we-test-cpus,5805.html (2024b), comprehensive CPU testing methodology emphasizing multiple iteration testing (minimum 3 runs), controlled test environments, and statistical consistency. Describes boot time measurement, application launch testing, and real-world workload benchmarking protocols used in industry-standard performance evaluation

Tom's Hardware Editorial Team(2023). Tom's Hardware Editorial Team: How we test: Benchmarking methodology and statistical validity. https://www.tomshardware.com/features/how-we-test-benchmarking (2023), detailed explanation of benchmarking statistical methods including average calculation, standard deviation analysis, and measurement consistency requirements. Establishes industry standards for performance testing repeatability and result validation

Tukey(1977). Tukey, J.W.: Exploratory Data Analysis. Addison-Wesley, Reading, MA (1977)